

Back To Practice

Q1

Save

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is allocated or not.
- If allocated, it displays the process number, process size, block number, block size, and the internal fragmentation (block size minus process size).
- If not allocated, it indicates that the process cannot be allocated.

Sample Input 1

```
5
100 500 200 300 600
4
212 417 112 426
```

Sample Output 1

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
```

Select Language: C (GCC 9.2.0)

```
2 int main() {
3     int m, n, i, j;
4     printf("Enter number of memory blocks:\n");
5     scanf("%d", &m);
6     int blockSize[m], originalBlock[m];
7     printf("Enter size of each block:\n");
8     for(i = 0; i < m; i++) {
9         scanf("%d", &blockSize[i]);
10        originalBlock[i] = blockSize[i];
11    }
12    printf("Enter number of processes:\n");
13    scanf("%d", &n);
14    int processSize[n];
15    printf("Enter size of each process:\n");
16    for(i = 0; i < n; i++) {
17        scanf("%d", &processSize[i]);
18    }
19    int allocation[n], fragment[n];
20    for(i = 0; i < n; i++) {
21        allocation[i] = -1;
22        fragment[i] = 0;
23    }
24    for(i = 0; i < n; i++) {
25        for(j = 0; j < m; j++) {
26            if(blockSize[j] >= processSize[i]) {
27                allocation[i] = j;
28                blockSize[j] -= processSize[i];
29                fragment[i] = blockSize[j];
30                break;
31            }
32        }
33    }
34    for(i = 0; i < n; i++) {
35        if(allocation[i] != -1) {
36            int b = allocation[i];
37            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
38                i + 1, processSize[i], b + 1, originalBlock[b], fragment[i]);
39        } else {
40            printf("Process %d of size %d is not allocated\n",
41                i + 1, processSize[i]);
42        }
43    }
44    return 0;
}
```

Test Cases 2

Custom Test Case

Run Sample Cases

Run All Cases

Back To Practice

< Q1 >

Save

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

- 1. The first line contains an integer m representing the number of memory blocks.
- 2. The next line contains m integers representing the sizes of each memory block.
- 3. The next line contains an integer n representing the number of processes.
- 4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Custom Test

Success

Input :

5
100 500 200 300 600
4
212 417 112 426

Expected Output :

Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated

Output :

Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183